

A Quantitative Comparison of Four Procedural Dungeon Generation Algorithms

Turing
RTSS Board
turingrtss@gmail.com

September 2026

Abstract

Procedural dungeon generation is a core technique in roguelike game development, yet comparative evaluations of generation algorithms using consistent metrics are rare. We implement four common approaches—Binary Space Partition (BSP) trees, cellular automata, random walk, and room placement with corridors—and evaluate them across 20 runs each on identical 80×40 grids. We measure five metrics: open space ratio, connectivity rate, room count, longest shortest path, and generation time. BSP trees and room placement achieve 100% connectivity with structured layouts. Cellular automata produce the most organic-looking caves but achieve 0% connectivity without post-processing. Random walk guarantees connectivity by construction but is the slowest algorithm (275 ms vs. 0.29 ms for room placement). We report all metrics, sample maps, and discuss the design trade-offs each algorithm implies.

1 Introduction

Procedural content generation (PCG) is fundamental to roguelike games, where each playthrough should present a unique dungeon layout [1]. The choice of generation algorithm affects not only visual aesthetics but also gameplay properties: room connectivity determines whether the player can reach all areas, open space ratio affects combat dynamics, and path lengths influence exploration time.

Despite the widespread use of procedural dungeon generation, quantitative comparisons of algorithms under controlled conditions are uncommon. Most discussions occur in blog posts, tutorials, and game development forums, where evaluation is typically visual (“this looks good”) rather than metric-based [2].

This paper implements four widely-used dungeon generation algorithms, evaluates them on five quantitative metrics over 20 runs each, and discusses the gameplay implications of each algorithm’s statistical profile.

2 Methods

2.1 Algorithms

We implement four dungeon generation algorithms, each representing a distinct approach to space partitioning:

Binary Space Partition (BSP) Trees. Recursively subdivide the map into rectangular regions, place a room within each leaf partition, and connect adjacent rooms with L-shaped corridors [3]. Parameters: minimum room size 5, maximum recursion depth 4.

Cellular Automata. Initialize a random grid (45% wall probability), then iterate a birth/-survival rule: a wall tile becomes floor if it has fewer than 5 wall neighbors; a floor tile becomes wall if it has 4 or more wall neighbors. 5 iterations [4].

Random Walk (Drunkard’s Walk). Start at the center, move in a random cardinal direction each step, carving floor tiles. Continue until 35% of the map is open [5].

Room Placement with Corridors. Place non-overlapping rectangular rooms at random positions (10 target rooms, size 4–9 tiles per dimension), then connect room centers sequentially with L-shaped corridors.

2.2 Map Configuration

All algorithms generate maps on an 80×40 character grid with a 1-tile solid border. The grid uses two tile types: wall (#) and floor (.).

2.3 Metrics

Each algorithm is evaluated on five metrics across 20 runs with seeds 0–19:

1. **Open space ratio:** fraction of tiles that are floor.
2. **Connectivity rate:** fraction of runs where all floor tiles are reachable from all other floor tiles via BFS.
3. **Room count:** number of distinct connected components of floor tiles.
4. **Longest shortest path:** BFS distance from a random floor tile to the farthest reachable tile. Indicates map traversal depth.
5. **Generation time:** wall-clock milliseconds per map.

3 Results

Table 1 summarizes all metrics across the four algorithms.

Table 1: Dungeon generation algorithm comparison (20 runs each, 80×40 grid).

Algorithm	Open %	Connected	Rooms	Path Len	Time (ms)
BSP Tree	0.421 ± 0.053	100%	1.0 ± 0.0	105 ± 17	0.88
Cellular Automata	0.558 ± 0.034	0%	15.2 ± 5.3	78 ± 28	52.80
Random Walk	0.350 ± 0.000	100%	1.0 ± 0.0	73 ± 20	274.72
Room Placement	0.189 ± 0.019	100%	1.0 ± 0.0	81 ± 22	0.29

3.1 Connectivity

Three of four algorithms achieve 100% connectivity. Cellular automata achieves 0%: across all 20 runs, the generated caves contain multiple disconnected regions (mean: 15.2 rooms). This is a well-known limitation of the cellular automata approach and is typically addressed by flood-filling the largest connected component or adding tunnels between components as a post-processing step [4].

3.2 Open Space

Cellular automata produces the most open maps (55.8%), while room placement produces the most compact (18.9%). The BSP and random walk algorithms fall in between at 42.1% and 35.0% respectively. Random walk achieves exactly its target ratio (35%) with zero variance, as it terminates precisely when the threshold is reached.

3.3 Path Length

BSP trees produce the longest shortest paths (105 steps), indicating deeper maps that require more exploration. This is a consequence of the tree structure: rooms are connected in a chain that follows the partition hierarchy. Cellular automata, despite having the most open space, has shorter paths (78 steps) because the disconnected caves limit traversable area.

3.4 Generation Speed

Room placement is the fastest algorithm at 0.29 ms per map—fast enough for real-time generation. BSP is similarly fast (0.88 ms). Cellular automata requires 52.8 ms due to 5 iterations over the full grid. Random walk is the slowest at 274.7 ms because it requires many steps to carve 35% of the map via single-tile moves.

4 Discussion

4.1 Algorithm Selection Guidelines

The choice of algorithm depends on the desired gameplay properties:

Structured dungeons (rooms connected by corridors): BSP trees or room placement. BSP produces more varied room sizes due to recursive subdivision; room placement gives more direct control over room count and size. Both guarantee connectivity.

Organic caves: Cellular automata, with mandatory post-processing for connectivity. The natural-looking layouts are visually distinct from rectangular rooms but require additional work to ensure playability.

Amorphous exploration spaces: Random walk produces connected, open areas without distinct rooms. Suitable for cave systems or natural terrain where room boundaries are undesirable.

Performance-critical applications: Room placement (0.29 ms) or BSP (0.88 ms). Either can generate thousands of maps per second, enabling real-time procedural generation.

4.2 Cellular Automata Connectivity

The 0% connectivity rate of cellular automata is the most significant finding. Any game using this algorithm must implement one of:

- Flood-fill to identify and connect components
- Tunnel-carving between the largest components
- Discarding disconnected maps and regenerating

Without post-processing, a cellular automata dungeon will contain areas the player cannot reach, potentially locking required items or exits behind walls.

4.3 Limitations

1. We evaluate only 2D grid-based generation. 3D and graph-based approaches are not covered.
2. The “room count” metric counts connected components, not architecturally meaningful rooms. BSP creates multiple rooms but connects them, yielding 1 component.
3. We do not evaluate aesthetic quality, which is subjective and game-dependent.
4. Parameters were not optimized per algorithm; different parameter settings would shift the metrics.

5 Conclusion

We compare four procedural dungeon generation algorithms on five quantitative metrics. BSP trees offer the best balance of structure, connectivity, path depth, and speed. Cellular automata produces visually organic caves but requires post-processing for connectivity. Random walk guarantees connectivity but is the slowest approach. Room placement is the fastest and most controllable but produces the most compact maps. All code and results are publicly available for reproduction.

Data and Code Availability

Code and results: <https://github.com/turingrtss/vulndetect>

References

- [1] T. Shaker, J. Togelius, and M. J. Nelson, *Procedural Content Generation in Games*, Springer, 2016.
- [2] H. Stoy, “Procedural Dungeon Generation Algorithm,” *Gamasutra*, 2015. [Online]. Available: <https://www.gamedeveloper.com/programming/procedural-dungeon-generation-algorithm>
- [3] S. Foley, “BSP Dungeon Generation,” in *Roguelike Development Resource*, 2014.
- [4] L. Johnson, “Cellular Automata Method for Generating Random Cave-Like Levels,” in *Roguebasin*, 2010. [Online]. Available: http://www.roguebasin.com/index.php/Cellular_Automata_Method_for_Generating_Random_Cave-Like_Levels
- [5] B. Nystrom, “Generating Dungeons with a Random Walk,” in *Journal of Game Development*, 2014.